Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Dersin ilk yarısında öğrencilerin Goldbach ödevi için yazdığı kodlar incelenir. Ardından büyük dizilerde alınan bellek hatası üzerinden bir prosesin bellek bölgeleri anlatılır. Algoritmaların nasıl karşılaştırılacağı konuşulur. Dersin sonunda "en büyük not" problemi üzerinden dizilere ve veri yapılarına neden ihtiyaç duyulduğu gösterilir.
- Goldbach ödevi
  - Asıl önerme (2'den büyük her çift sayı iki asalın toplamıdır) çözülmemiş bir problemdir. Ödevde önerme değiştirildi: "Girilen bir N sayısına kadar olan çift sayılar için önerme doğru mu?"
  - Öğrencilerin ortak yaklaşımı: asal sayıları bir dizide (asal "deposu") toplamak. Her çift sayı için asalları baştan hesaplamak yerine, daha önce bulunanlar saklanır ve yeni asallar eklenir. Bu bir performans kazancıdır.
  - Asallık testi için sayının karekökünе kadar bölmek yeterlidir. Karekökle uğraşmamak için `i * i <= sayi` koşulu kullanılabilir.
  - Asallık testinde yalnızca daha önce bulunmuş asal sayılara bölmek yeterlidir.
  - Çift bölenlere bakmaya gerek yoktur: 2'den sonraki asallar tektir. 4'ten büyük çift sayılarda 2 çıkarılınca kalan çift olur, asal olamaz. Bu yüzden 6'dan itibaren aramaya 3'ten başlanabilir.
  - Bir çift sayı için ilk uygun asal çift bulununca aramayı durdurmak yeterlidir. Bunun için bir bayrak (flag) değişkeni ve `while` kullanılır.
  - Bir öğrenci, sayıdan bir asal çıkardıktan sonra kalanın asal olup olmadığını sıralı asal dizisinde ikili arama (binary search) ile hızlıca kontrol etti.
  - Bir öğrenci asal dizisini en başta N'e kadar bir kerede doldurdu. Hoca, karşıt örnek daha küçük bir sayıda bulunursa fazladan asal hesaplanmış olacağını, ama önerme şu ana kadar çürütülmediği için pratikte bunun sonucu değiştirmediğini söyler. Adım adım doldurmayı daha doğru bulur.
  - Döngü şartıyla aynı bilgiyi taşıyan bir bayrak değişkeni gereksizdir. Örneğin bayrak yalnızca `i % j == 0` olunca sıfırlanıyorsa, `while (counter == 1)` yerine doğrudan `while (i % j != 0)` yazılabilir. Bir karşılaştırma ve bir değişken tasarruf edilir.
  - Trade-off: asalları dizide saklamak bellek harcar ama zaman kazandırır. Saklamamak bellekten kazandırır ama her sayı için asallar yeniden hesaplanır.
- Büyük dizilerde bellek hatası
  - Öğrenciler, asal dizisini çok büyük tanımladıklarında (ör. bir milyon eleman) programın hata verdiğini gördü. `static` yazınca veya `malloc` kullanınca sorun çözülüyordu.
  - Program çalıştırılınca proses olur. İşletim sistemi ona belleğin tamamını değil, belli bir bölgesini verir. Proses kontrol bloğu (process control block) ile yönetilir.
  - Prosesin bellek bölgeleri: program kodu, ilk değeri verilmiş değişkenler, ilk değeri verilmemiş değişkenler, yığın (stack) ve dinamik bellek (heap).
  - `int a[500];` gibi boyutu programda sabit yazılan diziler statik bellek ayırmadır (static allocation). `int *a = malloc(...)` ise dinamik bellek ayırmadır (dynamic allocation) ve heap'ten yer alır.
  - Hoca, sorunun dizinin istediği büyüklükte bitişik bir alanın verilememesinden kaynaklandığını söyler.
  - `static` anahtar kelimesi değişkeni stack dışında, daha geniş bir bölgede saklar. Bu bir çözümdür, ama "bıçak gibidir", özellikleri bilinmeden kullanılmamalıdır. Hoca şimdilik `static` kullanılmamasını ve küçük verilerle çalışılmasını önerir. Dinamik bel-

lek dönem sonunda veya gelecek dönem işlenecektir.
  - Bitişik alan gerektirmeyen bir veri yapısı da vardır: bağlı liste (linked list). Veriler bellekte dağınık durur ve birbirine bağlanır.
- Algoritmaları karşılaştırmak
  - Dört öğrencinin dört farklı Goldbach algoritması (A1, A2, A3, A4) nasıl karşılaştırılır?
  - Zaman ölçümü (timer): programın başlangıç ve bitiş zamanı arasındaki fark ölçülür. Ekrana yazdırma kapatılmalıdır.
  - Her öğrenci kendi bilgisayarında ölçerse sonuç anlamsızdır: işlemci, RAM, derleyici ve işletim sistemi farklıdır. Bütün kodlar aynı bilgisayarda, aynı N (ör. 100.000) ile çalıştırılmalıdır.
  - Aynı bilgisayarda bile işletim sistemi başka prosesler çalıştırır ve proseslerin önceliklerini değiştirir. Daha yavaş olması beklenen bir algoritma daha erken bitebilir. Bu yüzden zaman ölçümü (empirical analysis) kaba bir analizdir.
  - Daha objektif yöntem: temel işlem sayısına (number of basic operations) göre çalışma zamanı analizi (running time analysis, $T(N)$). Örneğin e^x probleminde iç içe döngüyü tek döngüye indirmek temel işlem sayısını azaltmıştı.
  - Bir öğrencinin sorusu üzerine: işletim sisteminin gösterdiği "Energy Impact" (büyük olasılıkla macOS'taki etkinlik ekranı) gibi değerler de bir etkidir, ama algoritma analizinde asıl ölçütler çalışma zamanı ve bellek kullanımıdır.
- C standardı uyarısı
  - `for (int i = 0; ...)` derste kullanılan Dev-C++ (TDM-GCC 4.8.1) ile derlenmez. Ekranda görülen hata: `'for' loop initial declarations are only allowed in C99 mode`. Döngü değişkeni yukarıda tanımlanmalıdır.
  - macOS'ta derleyici `int main()` yerine `int main(void)` isteyebilir. Bu, derleyicinin standart ayarıyla ilgilidir. Hoca kodların kendi yazdığı biçimde gönderilmesini ister.
- En büyük not problemi
  - N öğrencinin notu girilir ve en büyüğü bulunur. Değişkenler: `grade`, `N`, `max = 0`, `i`.
  - Her not girilince `if (grade > max) max = grade;`. Notlar 0 veya pozitif olduğu için `max = 0` ile başlamak güvenlidir.
  - Negatif not girilmesini önlemek için not okuma `do { ... } while (grade < 0);` içine alınır. Geçerli bir not girilene kadar aynı öğrencinin notu tekrar istenir. Bir öğrenci negatif değerin mutlak değerini almayı önerdi. Hoca bunu da kabul eder.
  - En büyük notun hangi öğrenciye ait olduğu `max_i = i` ile tutulur.
  - `>` kullanıldığı için eşit notlarda ilk öğrenci kalır. `>=` kullanılırsa son öğrenci alınır.
  - Döngü bittiğinde `i`'nin değeri N + 1'dir.
- En büyük ikinci not
  - Yeni not en büyükten büyükse: önce eski en büyük ikinci sıraya kaydırılır (`max2 = max; max2_i = max_i;`), sonra yeni değer en büyük olur (`max = grade; max_i = i;`).
  - Değilse ve en büyük ikinciden büyükse: yalnızca `max2 = grade; max2_i = i;`.
  - `max_i` ve `max2_i`'ye başlangıçta 0 verilmelidir, çünkü ilk notta `max2_i = max_i` ataması yapılır.
- Algoritmanın tıkandığı yer
  - En büyük 6. notu bulmak için `max1`, ..., `max6` ve bunların indisleri gerekir. 60. not istenirse 60 değişken ve uzun bir `if` zinciri gerekir. Çalışır ama "hamallıktır", programlama bu değildir.
  - Bir öğrencinin sorusu: en yüksek notu iki öğrenci aldıysa ikisi de yazılmalıdır. Notlar tek bir değişkenin üzerine yazıldığı için ilk girilen not kaybolur. Bu problem bu yaklaşımla çözülemez.
  - Algoritmanın üç temel yapısı bu problemleri çözmeye yetmez. Girilen verilerin bellekte saklanması gerekir. Bunu programlama dili sağlar.
- Veri soyutlama (data abstraction) ve veri yapıları
  - Veri yapıları, verinin bellekte (özellikle RAM'de) nasıl organize edileceğini belirleyen mantıksal yapılardır.
  - Dizi: bellekte ardışık hücrelerdir. `int a[10]` için `a`, 0. elemanın adresini tutar. `a[i]` ile her elemana ulaşılır. Baştan sona sırayla erişim sıralı (sequential) erişimdir, `a[5]` gibi doğrudan erişim rastgele (random) erişimdir. Arka planda indis ile bir adres hesabı yapılır.
  - Notlar bir dizide saklanırsa: önce en büyük değer bulunur, sonra dizi bir kez daha gezilerek bu değere sahip bütün öğrenciler yazdırılır. Öğrencinin sorduğu problem böyle çözülür.

- En büyük k. değer için bütün diziyi sıralamak bir çözümdür, ama daha iyisi vardır: yalnızca ilk k elemanı bulmak veya hızlı sıralama algoritmaları (özyinelemeli algoritmalar konusunda görülecektir).
- Matris: iki boyutlu veri. Örneğin satırlar öğrenciler, sütunlar vize, final, ödev ve ortalama. Bellekte yine tek boyutlu, ardışık saklanır. `M[i][j]` erişimi bir adres hesabıdır. Bu, pointer konusunda anlatılacaktır.
- Graf: şehirler ve aralarındaki yollar gibi bağlantılar. Örneğin 9 şehir için 9×9'luk bir uzaklık matrisi saklanır (yollar iki yönlüyse simetriktir). Bu veri üzerinde en kısa yol algoritması çalıştırılır.
- Ağaç: rektör, dekanlar, bölümler gibi hiyerarşik yapılar. Düzenli bir ağaç dizide saklanabilir. Düzensizse bağlı liste mantığı kullanılır.
- Bütün veri yapılarının temeli dizilerdir ve matrislerdir. Bu derste diziler, matrisler ve bağlı listeler görülecektir.
- Programlamada RAM modeli kullanılır: belleğin istenen hücresine doğrudan erişilir. Otomata teorisindeki modellerde (deterministik ve deterministik olmayan sonlu otomatlar, yığıtlı otomat, Turing makinesi) belleğe erişim kısıtlıdır. Hoca bunların araştırılmasını önerir. İkinci sınıfta bunun dersi vardır (büyük olasılıkla Hesaplama Kuramı).

Hocanın Özellikle Vurguladığı Kısımlar

- Döngü değişkenini `for`'un içinde tanımlamayın. Derste kullanılan derleyici ayarında (büyük olasılıkla C89/C90) bu hata verir.
- Asalları saklamak ile her seferinde yeniden hesaplamak arasında bellek-zaman ödünleşimi vardır.
- İki algoritma, farklı bilgisayarlarda ölçülen sürelerle karşılaştırılmaz. Zaman ölçümü bile kaba bir analizdir. Asıl ölçüt temel işlem sayısıdır.
- Bir veri giriş işleminden önce mutlaka `printf` ile kullanıcıya ne gireceği söylenmelidir.
- Fonksiyon kullanmak programı hızlandırmaz, modüler hâle getirir.
- Mesleki derslerde not için değil, "ileride bunu birine öğretebilirim" düşüncesiyle çalışın. Not kendiliğinden gelir.

Kısa Tekrar Notları

- Goldbach: asal deposu (dizi) + bayrak + `while`. Asallık için karekök sınırı: `i * i <= n`.
- Bayrak, döngü şartıyla aynı bilgiyi taşıyorsa gereksizdir.
- Bellek bölgeleri: kod, ilk değerli veriler, ilk değersiz veriler, stack, heap.
- Statik dizi: `int a[500];`. Dinamik: `malloc`.
- Algoritma karşılaştırma: aynı makine + aynı N. Daha iyisi: temel işlem sayısı.
- En büyük: `if (grade > max) { max = grade; max_i = i; }`.
- En büyük iki: önce eskiyi `max2`'ye kaydır, sonra güncelle. Değilse `max2` ile karşılaştır.
- Geçerli giriş için `do { scanf(...); } while (grade < 0);`.
- k. en büyük, eşit notlar: diziler gerekir.
- Veri yapıları: dizi, matris, bağlı liste, ağaç, graf.

Detaylı Açıklamalar

En büyük iki not programı

Derste yazılan programın son hâli (ekran görüntüsünden):

```c
#include <stdio.h>
int main(){
    int grade,N,max=0,max2=0,max_i,max2_i,i;
    printf("Ogrenci sayisi=");
    scanf("%d",&N);
    max_i=max2_i=0;
    for(i=1;i<=N;i++){
        do{
            printf("%d. ogrencinin notu",i);
```

```
            scanf("%d",&grade);
        }while(grade<0);
        if(grade>max){
            max2=max;
            max2_i=max_i;
            max=grade;
            max_i=i;
        }else if(grade>max2){
            max2=grade;
            max2_i=i;
        }
    }
    printf("Max Not=%d, bu not %d. ogrencinindir",max,max_i);
    printf("Max_2 Not=%d, bu not %d. ogrencinindir",max2,max2_i);
    printf("%d",i);
    return 0;
}
```

Derste 5 öğrenci için 3, 4, 2, 1, 5 notları girildi. Çıktı: en büyük not 5 (5. öğrenci), en büyük ikinci not 4 (2. öğrenci), döngü sonunda `i` = 6. Ders sonrasında kod `gcc -Wall` ile derlendi ve aynı girdiyle aynı sonucu verdi.

Not: Bu kodda en büyük nota eşit bir not gelirse `grade > max` yanlış, `grade > max2` doğru olur. Bu durumda eşit not en büyük ikinci olarak kaydedilir. Yani iki öğrenci aynı en yüksek notu aldıysa en büyük ikinci not da aynı değer olur.

Ders sonrası notlar

- **Büyük dizi hatasının asıl sebebi:** Derste hata, istenen büyüklükte bitişik alan bulunamamasıyla açıklandı ve stack bölgesinde "dinamik dizilerin" saklandığı söylendi. Doğrusu şöyledir: Bir fonksiyonun içinde tanımlanan `int a[N];` gibi yerel diziler stack'te saklanır. Stack'in boyutu sınırlıdır ve program başlarken sabittir. Bu sınır büyük olasılıkla Windows'ta 1-2 MB, Linux'ta 8 MB dolayındadır. 518.000 elemanlı bir `int` dizisi yaklaşık 2 MB eder. Öğrencinin bu sınırda hata alması büyük olasılıkla Windows'taki stack sınırıyla uyumludur. Sorun stack'in dolmasıdır (stack overflow).
- **`static` ve `malloc` neden çözüyor:** `static int a[N];` veya fonksiyon dışında (global) tanımlanan diziler stack'te değil, statik veri bölgesinde (ilk değeri verilmemişse BSS bölgesinde) saklanır. Bu bölgenin stack gibi küçük bir sınırı yoktur. `malloc` ile ayrılan bellek ise heap'tendir (dinamik bellek). Heap de çok büyük olabilir. Bitişik alan gereksinimi her iki durumda da vardır, ama sanal bellek sayesinde pratikte büyük bir sorun oluşturmaz.
- **Bayrak yerine şart:** Hocanın önerdiği değişiklik (`while (counter == 1)` yerine `while (i % j != 0)`), öğrencinin kodunda bayrak döngüden sonra da kullanıldığı için doğrudan uygulanamadı. Bayrak döngü dışında başka bir amaçla kullanılıyorsa kaldırılmamalıdır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.